Investigación de Persistencia de Datos en Android y Modelo Entidad-Relación: CityReport

En el desarrollo moderno en Android con Kotlin y Jetpack Compose, la persistencia de datos local se divide en dos grandes enfoques: el almacenamiento de pares clave-valor (configuraciones ligeras, preferencias de usuario, estados de sesión) y las bases de datos relacionales estructuradas (registros complejos con relaciones e integridad referencial).

1. Almacenamiento Clave-Valor: SharedPreferences vs. Jetpack DataStore

1.1 ¿Cómo funciona cada uno?

- **SharedPreferences:** API tradicional presente desde las primeras versiones de Android que guarda datos primitivos en un archivo XML dentro del almacenamiento interno de la aplicación. Opera cargando todo el archivo en memoria y ofrece lecturas síncronas y escrituras mediante `apply()` (asíncrono en memoria, pero bloquea el hilo principal en los ciclos de vida como `onStop()` o `onPause()`) o `commit()` (síncrono directo a disco).
- **Jetpack DataStore:** Solución moderna de almacenamiento de datos basada en **Kotlin Coroutines y Flow**. Ofrece dos implementaciones: *Preferences DataStore* (almacena pares clave-valor sin esquema previo, similar a SharedPreferences) y *Proto DataStore* (almacena instancias de tipos de datos personalizados con tipado fuerte mediante Protocol Buffers). Todas las operaciones de lectura y escritura ocurren estrictamente en un hilo secundario (`Dispatchers.IO`).

1.2 Comparativa Técnica

Atributo	SharedPreferences	Preferences DataStore	Proto DataStore
API Asíncrona	Parcial (Listeners básicos)	Total (Flow / Coroutines)	Total (Flow / Coroutines)
Hilo Principal (UI)	Bloquea en lecturas y en fsync	100% libre de bloqueos (ANR)	100% libre de bloqueos (ANR)
Seguridad de Tipos	Nula (solo primitivos sueltos)	Parcial (Preferences.Key<T>)	Estricta (Esquema .proto)
Manejo de Errores	Lanza excepciones en runtime	Emite errores vía Flow.catch	Emite errores vía Flow.catch
Transaccionalidad	Inconsistente en multihilo	Atómica (bloques edit {})	Atómica (updateData {})

1.3 Ventajas y Desventajas en Android

- **SharedPreferences:**
  - **Ventajas:** Rápido de implementar en proyectos legacy; no requiere dependencias externas en Gradle ni conocimientos de programación reactiva.
  - **Desventajas:** Provoca errores *Application Not Responding* (ANR) al bloquear el hilo principal (UI) esperando confirmaciones de escritura en disco (`fsync`); carece de manejo seguro de excepciones de I/O (`IOException`) y no garantiza consistencia transaccional en accesos concurrentes.
- **Jetpack DataStore:**

- **Ventajas:** Totalmente reactivo (ideal para observar estados directamente en **Jetpack Compose** mediante `collectAsStateWithLifecycle()`); garantiza consistencia atómica tipo ACID ligero; incluye soporte nativo de migración automática desde `SharedPreferences`.
- **Desventajas:** Curva de aprendizaje mayor si no se dominan Coroutines y Flows; *Proto DataStore* requiere configurar el compilador de Protocol Buffers y definir archivos `.proto`.

---

2. Bases de Datos Relacionales: SQLite Directo vs. Room

2.1 ¿Cómo funciona cada uno?

- **SQLite Directo (`SQLiteOpenHelper`):** Motor de base de datos relacional embebido nativo de Android. El desarrollador interactúa directamente con la API de bajo nivel escribiendo consultas en cadenas de texto planas (`rawQuery`, `execSQL`, `ContentValues`), gestionando manualmente la apertura y cierre de conexiones y recorriendo objetos `Cursor` fila por fila para instanciar objetos de Kotlin.
- **Room Persistence Library:** Capa de abstracción oficial sobre SQLite perteneciente a Android Jetpack. Utiliza anotaciones (`@Entity`, `@Dao`, `@Database`, `@Relation`) procesadas en tiempo de compilación (mediante **KSP**). El compilador genera automáticamente el código boilerplate de SQLite, valida la sintaxis SQL en tiempo de compilación y mapea los resultados directamente a `data class` o flujos reactivos (`Flow`).

2.2 Comparativa Técnica

Atributo	SQLite Directo (SQLiteOpenHelper)	Room (Jetpack)
Validación de SQL	En tiempo de ejecución (crashes si hay errores de sintaxis)	En tiempo de compilación (falla el build antes de ejecutar)
Mapeo de Objetos	Manual mediante índices de Cursor	Automático a data class de Kotlin
Código Boilerplate	Alto (contratos, cursores, bloques try/catch)	Mínimo (interfaces @Dao y anotaciones)
Reactividad y UI	Manual (requiere ContentObserver propio)	Nativa (Flow<List<T>>, LiveData, Paging 3)
Migraciones	Manuales y propensas a corrupción de esquema	Estructuradas (Migration(1, 2) y AutoMigrations)

2.3 Ventajas y Desventajas en Android

- **SQLite Directo:**
  - **Ventajas:** Cero sobrecarga de librerías externas; control total sobre consultas dinámicas ad-hoc donde la estructura de las tablas no se conoce en tiempo de compilación.

    - **Desventajas:** Altísimo riesgo de fugas de memoria (*memory leaks*) por cursores no cerrados; si se renombra una columna y se olvida actualizar una consulta en texto plano, la aplicación falla en producción al abrir esa pantalla; no impide por defecto ejecutar consultas pesadas en el hilo principal.
- **Room:**
    - **Ventajas:** Verificación de esquemas y consultas en compilación; integración nativa con funciones `suspend` y `Flow` (la interfaz en Jetpack Compose se recompone automáticamente cuando cambia un registro en la base de datos); facilita las pruebas unitarias e instrumentadas con bases de datos en memoria (`Room.inMemoryDatabaseBuilder`).
    - **Desventajas:** Añade un ligero incremento en el tiempo de compilación por el procesamiento de anotaciones (reducido notablemente al utilizar KSP en lugar de KAPT).

---

3. Modelo Entidad-Relación (ER) para CityReport

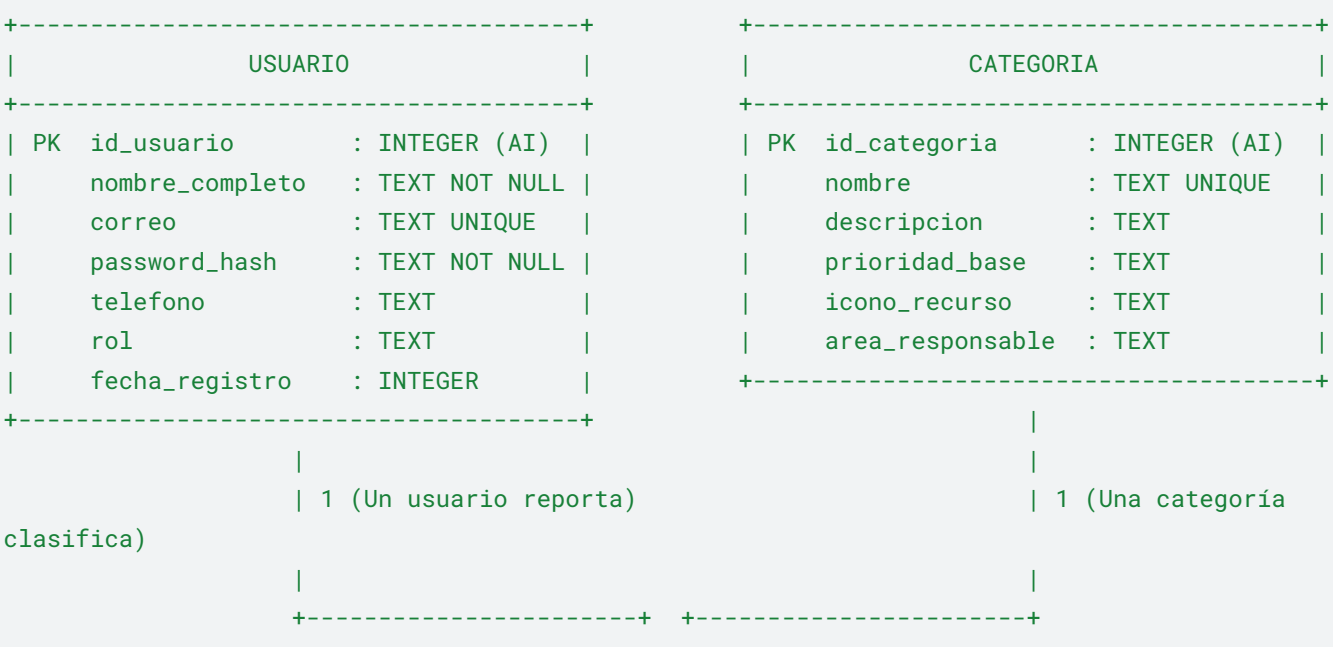
El modelo de datos de **CityReport** está diseñado para soportar el reporte ciudadano de incidencias urbanas integrando captura fotográfica local (**CameraX**), geolocalización (**FusedLocationProviderClient**) y persistencia estructurada en **Room**.

3.1 Reglas de Negocio y Relaciones (Cardinalidad)

1. **Relación USUARIO — INCIDENCIA (1:N):** Un **Usuario** puede reportar cero o muchas **Incidencias** (`0..N`), pero cada **Incidencia** es registrada por un único **Usuario** (`1..1`). La clave primaria `id_usuario` viaja como clave foránea (`FK`) a la tabla `INCIDENCIA`.
2. **Relación CATEGORIA — INCIDENCIA (1:N):** Una **Categoría** (ej. *Bacheo, Alumbrado Público, Fuga de Agua, Recolección de Basura*) clasifica cero o muchas **Incidencias** (`0..N`), pero cada **Incidencia** pertenece a una única **Categoría** principal (`1..1`). La clave primaria `id_categoria` viaja como clave foránea (`FK`) a la tabla `INCIDENCIA`.

3.2 Diagrama Visual Entidad-Relación (Esquema Estructural)

None



```
                    |  |
                    v  v
                  0..N (Muchas incidencias)
+-------------------------------------------------+
|                   INCIDENCIA                    |
+-------------------------------------------------+
| PK  id_incidencia        : INTEGER (AI)         |
| FK  id_usuario           : INTEGER (Ref USUARIO) |
| FK  id_categoria         : INTEGER (Ref CATEG.)  |
|     titulo               : TEXT NOT NULL        |
|     descripcion          : TEXT NOT NULL        |
|     latitud              : REAL NOT NULL        |
|     longitud             : REAL NOT NULL        |
|     direccion_referencia : TEXT                 |
|     foto_uri             : TEXT                 |
|     estado               : TEXT NOT NULL        |
|     fecha_reporte        : INTEGER NOT NULL     |
|     fecha_actualizacion  : INTEGER              |
|     sincronizado         : INTEGER (Boolean)    |
+-------------------------------------------------+
```

3.3 Código Mermaid del Diagrama ER (Para el Repositorio en GitHub)

Copia y pega este bloque dentro de un archivo Markdown (por ejemplo, `README.md` o `docs/ER_Diagram.md`) en el repositorio de GitHub para que se renderice automáticamente el diagrama ER:

```
None
erDiagram
    USUARIO ||--o{ INCIDENCIA : "reporta (1:N)"
    CATEGORIA ||--o{ INCIDENCIA : "clasifica (1:N)"

    USUARIO {
        int id_usuario PK "Auto-incremental"
        string nombre_completo "NOT NULL"
        string correo "UNIQUE, NOT NULL"
        string password_hash "NOT NULL"
        string telefono "Opcional"
        string rol "CIUDADANO | ADMIN"
        long fecha_registro "Timestamp (Epoch ms)"
    }

    CATEGORIA {
        int id_categoria PK "Auto-incremental"
        string nombre "UNIQUE, NOT NULL (ej. Bache, Alumbrado)"
        string descripcion "Detalle del tipo de reporte"
        string prioridad_base "BAJA | MEDIA | ALTA"
        string icono_recurso "Identificador de icono UI"
        string area_responsable "Dependencia municipal"
    }
```

```
INCIDENCIA {
    int id_incidencia PK "Auto-incremental"
    int id_usuario FK "Ref -> USUARIO.id_usuario"
    int id_categoria FK "Ref -> CATEGORIA.id_categoria"
    string titulo "Resumen corto del problema"
    string descripcion "Detalle del incidente"
    double latitud "Coordenada GPS (FusedLocation)"
    double longitud "Coordenada GPS (FusedLocation)"
    string direccion_referencia "Calle, colonia o cruce"
    string foto_uri "Ruta local o URI de CameraX"
    string estado "PENDIENTE | EN_PROCESO | RESUELTO | RECHAZADO"
    long fecha_reporte "Timestamp de creación"
    long fecha_actualizacion "Timestamp de último cambio"
    boolean sincronizado "Flag para modo offline-first"
}
```

---

4. Diccionario de Datos Detallado

4.1 Tabla: `USUARIO`

Representa a los ciudadanos o administradores registrados en la aplicación. (Nota arquitectónica: el estado de la sesión activa como user_id_activo o auth_token se almacena en Preferences DataStore, mientras que los datos del perfil residen en Room).

Campo	Tipo (SQLite / Kotlin)	Restricciones	Descripción
id_usuario	INTEGER / Int	PK, AutoGenerate	Identificador único del usuario.
nombre_completo	TEXT / String	NOT NULL	Nombre y apellidos del ciudadano.
correo	TEXT / String	UNIQUE, NOT NULL	Correo electrónico usado para iniciar sesión.
password_hash	TEXT / String	NOT NULL	Hash de contraseña para validación de acceso.
telefono	TEXT / String?	NULLABLE	Número de contacto opcional para seguimiento.
rol	TEXT / String	NOT NULL	Rol del perfil (CIUDADANO o ADMIN).
fecha_registro	INTEGER / Long	NOT NULL	Fecha de alta en milisegundos (Epoch timestamp).

4.2 Tabla: `CATEGORIA`

Catálogo normalizado de tipos de desperfectos o servicios urbanos para evitar redundancia de texto y facilitar el filtrado en la interfaz.

Campo	Tipo (SQLite / Kotlin)	Restricciones	Descripción
id_categoria	INTEGER / Int	PK, AutoGenerate	Identificador único de la categoría.
nombre	TEXT / String	UNIQUE, NOT NULL	Nombre de la categoría (ej. Bacheo, Alumbrado, Agua).
descripcion	TEXT / String	NOT NULL	Explicación de qué incidentes abarca la categoría.
prioridad_base	TEXT / String	NOT NULL	Nivel de urgencia por defecto (BAJA, MEDIA, ALTA).
icono_recurso	TEXT / String	NOT NULL	Nombre o identificador del icono vectorial en Compose.
area_responsable	TEXT / String	NOT NULL	Área municipal encargada (ej. Obras Públicas, SAPASA).

4.3 Tabla: `INCIDENCIA`

Entidad central de **CityReport** que almacena cada reporte ciudadano con su evidencia fotográfica, coordenadas GPS y estado de atención.

Campo	Tipo (SQLite / Kotlin)	Restricciones	Descripción
id_incidencia	INTEGER / Int	PK, AutoGenerate	Folio o identificador único del reporte.
id_usuario	INTEGER / Int	FK (USUARIO), INDEX	Usuario que creó el reporte (ON DELETE CASCADE).
id_categoria	INTEGER / Int	FK (CATEGORIA), INDEX	Categoría asignada al reporte (ON DELETE RESTRICT).
titulo	TEXT / String	NOT NULL	Encabezado breve del problema reportado.
descripcion	TEXT / String	NOT NULL	Explicación detallada de la incidencia.
latitud	REAL / Double	NOT NULL	Latitud GPS obtenida con FusedLocationProviderClient.
longitud	REAL / Double	NOT NULL	Longitud GPS obtenida con FusedLocationProviderClient.

Campo	Tipo (SQLite / Kotlin)	Restricciones	Descripción
direccion_referencia	TEXT / String?	NULLABLE	Calle, número, colonia o referencias visuales.
foto_uri	TEXT / String	NOT NULL	Ruta local o Content URI de la foto tomada con CameraX.
estado	TEXT / String	NOT NULL	Estado actual (PENDIENTE, EN_PROCESO, RESUELTO, RECHAZADO).
fecha_reporte	INTEGER / Long	NOT NULL	Timestamp del momento exacto en que se creó el reporte.
fecha_actualizacion	INTEGER / Long	NOT NULL	Timestamp del último cambio de estado.
sincronizado	INTEGER / Boolean	NOT NULL (Default 0)	Indica si el registro local ya se sincronizó con el servidor.